Operating Systems Project — Review 2

Methodology and Partial Implementation (Lab Assessment 9)

Event-Driven Runtime-Adaptive CPU Scheduling with Sequential Tabular Q-Learning

- **Repository:** github.com/lostpilot404/OS-Project
- **Language and libraries:** Python 3.11.2 with NumPy 2.4.6, pandas 3.0.6 and pytest 9.1.1
- **Automated verification:** 273 checks, all passing (re-run during this build)
- **Dataset:** 7 synthetic families: 6,000 training episodes, 140 validation workloads, 210 held-out test workloads
- **Date:** 11 October 2026

1. Detailed methodology of the proposed system

The proposed system is a single-CPU, event-driven, runtime-adaptive CPU scheduler. Instead of committing one policy for the lifetime of a workload, it decides at every dispatch or Round-Robin quantum boundary which of four classical policies (FCFS, SJF, Round Robin, Priority) governs the next execution segment of the same evolving trace. The controller sees only work that has arrived, so every measured gain is attributable to information a real scheduler could have had.

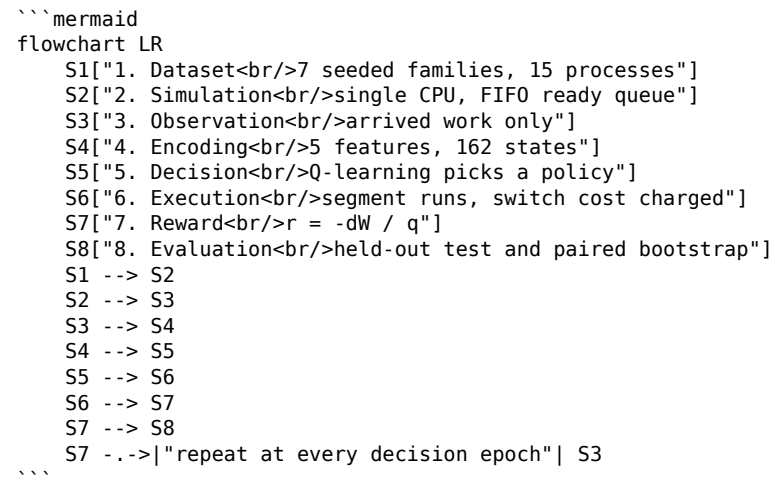
1.1 Research questions

- RQ1 — Can a single-CPU scheduler switch between FCFS, SJF, Round Robin and Priority at run time, using only information available at that instant?
- RQ2 — Does such switching reduce mean waiting time relative to the fixed policies it switches between, once a context-switch cost is charged?
- RQ3 — Does a learned switching policy add value over a hand-written adaptive rule that sees exactly the same observation?
- RQ4 — Do any gains survive on a workload family that was never seen during training, and are they larger than the paired bootstrap noise?

1.2 Methodology pipeline

The method is a closed loop between a discrete-event environment that owns the future and a controller that may see only the past and the present. Stages 3 to 7 run once per decision epoch inside a single trace; stage 8 runs once per experiment.

Diagram 1 — Methodology pipeline (section 1.2)



```mermaid
flowchart LR
    S1["1. Dataset<br/>7 seeded families, 15 processes"]
    S2["2. Simulation<br/>single CPU, FIFO ready queue"]
    S3["3. Observation<br/>arrived work only"]
    S4["4. Encoding<br/>5 features, 162 states"]
    S5["5. Decision<br/>Q-learning picks a policy"]
    S6["6. Execution<br/>segment runs, switch cost charged"]
    S7["7. Reward<br/>r = -dW / q"]
    S8["8. Evaluation<br/>held-out test and paired bootstrap"]
    S1 --> S2
    S2 --> S3
    S3 --> S4
    S4 --> S5
    S5 --> S6
    S6 --> S7
    S7 --> S8
    S7 -.->|"repeat at every decision epoch"| S3
```

- **Stage 1 — Dataset —** workload/generator.py: generate seven seeded families of 15-process workloads with SHA-256 derived seeds. Output: workload list and manifest.
- **Stage 2 — Simulation —** scheduler/runtime.py: advance one CPU through arrivals, one FIFO ready queue, remaining bursts and switch cost. Output: decision epochs and schedule trace.
- **Stage 3 — Observation —** scheduler/runtime.py: freeze an immutable snapshot of arrived and ready work only. Output: RuntimeObservation.
- **Stage 4 — Encoding —** rl/runtime_state.py: discretise five causal features into 162 tabular states (3 x 3 x 3 x 3 x 2). Output: state index s in [0, 162).
- **Stage 5 — Decision —** rl/runtime_controller.py: epsilon-greedy during training and greedy during evaluation over four policy actions, with an explicit fallback in unseen states. Output: action a in {FCFS, SJF, Round Robin, Priority}.
- **Stage 6 — Execution —** scheduler/runtime.py: run the selected segment (Round Robin bounded by the quantum), charge one unit when the running PID changes, settle endpoint arrivals. Output: advanced simulated time.
- **Stage 7 — Reward —** scheduler/runtime.py: measure the waiting-time area dW accumulated by ready processes and return r = -dW / q with gamma = 1. Output: interval reward r_t.
- **Stage 8 — Evaluation —** experiments/runtime_experiment.py: greedy, read-only runs on held-out workloads; paired differences with a family-stratified crossed bootstrap. Output: metric tables and 95% intervals.

1.3 Causal observation contract

The environment privately owns the complete event list so that it can admit future arrivals at their arrival times. The controller receives only an immutable RuntimeObservation, and the encoder and controller interfaces accept no Workload object.

Included in the observation:

- current simulated time and the policy selected at the immediately preceding epoch (None at the first epoch);
- processes that have arrived and are currently ready, with PID, arrival time, known burst, remaining burst and priority;
- counts of arrived and completed processes;
- aggregates restricted to work that has already arrived: observed mean burst and priority spread;

- mean arrival age of currently ready jobs (current time minus arrival time, which includes earlier CPU service and is not accumulated queue waiting).

Deliberately excluded:

- any data about processes that have not arrived yet, including their future bursts, priorities and arrival times;
- the total size of the workload and the identity of its family;
- counterfactual or completed-schedule metrics of policies that were not run.

A paired hidden-future test changes the arrival, burst and priority of unarrived processes while keeping the observable prefix fixed, and asserts that the observation and action prefixes remain identical.

Burst assumption: exact burst lengths are known once a process has arrived. This is the same assumption the standalone SJF model already requires; no burst predictor is included.

1.4 State abstraction and learning objective

The encoder maps each observation to one of 3 x 3 x 3 x 3 x 2 = 162 states, with quantum q = 4:

- **ready_count —** 1; 2–3; 4 or more; why: ready-queue contention.
- **completed_count —** 0; 1–3; 4 or more; why: trace progress, without knowing the workload size.
- **median_ready_remaining_burst —** at most q; at most 4q; more than 4q; why: remaining service demand of ready jobs.
- **mean_ready_arrival_age —** at most q; at most 4q; more than 4q; why: age of ready jobs, including earlier service.
- **arrived_work_priority_spread —** zero; non-zero; why: whether arrived processes have distinct priorities.

Each trace is one sequential episode. Between consecutive decision epochs the environment measures the waiting-time area dW accumulated by ready processes, including any switch interval, and returns the scaled interval reward:

```
r_t = -dW_t / max(1, q)        gamma = 1.0
sum over t of r_t = -TotalWaitingTime / q
```

Because gamma = 1 and every episode is finite, maximising the undiscounted return is equivalent to minimising total waiting time. The tabular update is:

```
Q(s, a) <- Q(s, a) + alpha * ( r_t + gamma * max over visited a' of Q(s', a') - Q(s, a) )
alpha = 0.1; terminal transitions omit the bootstrap term; Q initialised to 0
```

Exploration: epsilon starts at 1.0, is multiplied by 0.997 after each episode and is floored at 0.05. The first visit to a state forces a uniformly random action. Greedy choices and bootstrap targets consider only actions with a positive visit count, and ties go to the lowest action index. A wholly unseen state in evaluation triggers an explicit, counted fallback to the predeclared heuristic rather than a silent zero-value tie.

1.5 Methodological controls

- **Fixed-policy baselines —** FCFS, SJF, Round Robin and Priority run on identical workloads and pay the identical switch cost
- **Predeclared adaptive rule —** a non-learning heuristic with no fitted parameters, fixed in the configuration before the held-out test is run, so learning is not credited for what fixed rules already achieve

- **Blinded held-out split —** the final test (master seed 19301) is generated after training and validation, and the pipeline asserts zero fingerprint overlap with training and validation
- **Retired test seed —** an earlier final-test seed (9301) was examined during an independent audit and informed the queue and reward remediation (section 9); it is retired as a final claim, and no result from it is reported
- **Out-of-family generalisation —** poisson_arrivals is excluded from training entirely and used only at evaluation time
- **Independent seeds —** five agents (seeds 7101–7105) are trained separately, and Q-learning results average all five
- **Paired uncertainty —** differences are paired per workload and per model seed, then resampled with a family-stratified crossed bootstrap of 1,000 replicates (seed 104729)
- **Determinism —** evaluation is greedy and read-only, and every artifact regenerates from the declared seeds
- **Separation of timing —** observation, selection, Q-update and simulator wall time are recorded in separate columns and never charged to simulated time

2. Overall conceptual, architectural and pipeline diagram

The system is layered so that the information boundary is structural: the environment owns the future and the accounting, the controller receives a frozen snapshot of arrived work, and the controller's reply is a single policy name. The controller cannot modify the queue, the bursts or the event list.

Diagram 2 — System architecture (section 2)

```mermaid
flowchart TB
    subgraph ENV["Environment: scheduler/runtime.py (owns the future)"]
        E1["Private event list: future arrivals"]
        E2["Single FIFO ready queue and remaining bursts"]
        E3["CPU segment, switch cost, schedule trace"]
    end
    subgraph CTL["Controller: arrived work only"]
        C1["RuntimeObservation (frozen snapshot)"]
        C2["State encoder: 162 states"]
        C3["Q-learning controller: 648 state-action pairs"]
    end
    M["Waiting-time increment gives reward r = -dW / q"]
    E1 --> E2
    E2 --> E3
    E3 -->|"observation"| C1
    C1 --> C2
    C2 --> C3
    C3 -->|"action: FCFS, SJF, RR or Priority"| E3
    E3 --> M
    M -.->|"Q update"| C3
```

- **Data layer —** workload/models.py, workload/generator.py: defines Process and Workload objects and generates the seven seeded families.
- **Simulation layer —** scheduler/runtime.py: discrete-event loop, FIFO ready queue, remaining bursts, switch cost and schedule trace.
- **Baseline layer —** scheduler/fcfs.py, sjf.py, round_robin.py, priority.py: standalone fixed policies, kept so runtime actions can be checked against them.
- **Perception layer —** rl/runtime_state.py: encodes an immutable observation into one of 162 causal states.

- **Decision layer —** rl/runtime_controller.py, rl/runtime_heuristic.py: the learned tabular Q-learning policy and the predeclared non-learning rule.
- **Experiment layer —** experiments/runtime_experiment.py, experiments/runtime_config.py: splits, training, validation, held-out testing, paired bootstrap and artifact export.
- **Evaluation layer —** evaluation/metrics.py: metric definitions for waiting, turnaround, response, utilisation, throughput and switches.

2.1 Runtime control flow

1. The environment advances to the next decision epoch (a dispatch point, a completion or a Round-Robin quantum boundary) and admits every arrival at or before that instant, in (arrival time, pid) order.
2. It freezes an observation and passes it to the controller, which returns one of four policy names.
3. If the selected process differs from the running PID, one unit of switch cost is charged before the segment starts. Arrivals during that interval are queued, but the chosen dispatch is not reconsidered.
4. The segment runs: Round Robin is bounded by the quantum, the other policies run to completion. Endpoint arrivals are admitted first, then an unfinished Round-Robin process is appended to the tail.
5. The environment measures the waiting-time area accumulated by ready processes, converts it to the interval reward and returns it to the controller for the Q update.
6. A policy switch changes only the next selection rule and service length. The queue, the remaining bursts and the switch accounting carry over unchanged.

3. Description of the major modules

Six modules carry the system. Two build and run the simulated machine (M1, M2), two implement the learned controller (M3, M4), one is the non-learning reference rule (M5), and one orchestrates the experiment and its statistics (M6). Section 4 gives the workflow diagram for each.

M1 — Workload generator and dataset builder

- **Files:** workload/generator.py, workload/models.py
- **Contract:** family configuration and seed → validated workload with fingerprint
- **Responsibility:** Generates the seven synthetic families and validates every process (non-negative PID, arrival and priority; positive burst; unique PIDs). Records split, family, seed and fingerprint metadata.
- **Status:** implemented and covered by automated tests

M2 — Discrete-event runtime environment

- **Files:** scheduler/runtime.py
- **Contract:** workload, controller, quantum and switch cost → schedule trace and six metrics
- **Responsibility:** Owns the private event list, the single FIFO ready queue, remaining bursts, switch-cost accounting, the causal observation and the interval reward.
- **Status:** implemented and covered by automated tests

M3 — Causal observation and state encoder

- **Files:** rl/runtime_state.py
- **Contract:** RuntimeObservation → state index in [0, 162)
- **Responsibility:** Extracts five causal features, discretises them relative to the quantum and flattens them in mixed-radix order. It accepts an observation only and rejects any other object, including a Workload.
- **Status:** implemented and covered by automated tests

M4 — Sequential Q-learning controller

- **Files:** rl/runtime_controller.py, rl/q_learning.py
- **Contract:** observation (encode) and reward (update) → policy action; Q table and visit counts
- **Responsibility:** Tabular Q-learning over 162 x 4 = 648 state-action pairs, with epsilon-greedy exploration, first-visit forcing, unvisited-action masking, read-only greedy evaluation and a counted heuristic fallback.
- **Status:** implemented and covered by automated tests

M5 — Predeclared heuristic controller

- **Files:** rl/runtime_heuristic.py
- **Contract:** RuntimeObservation → policy action
- **Responsibility:** Non-learning rule (Round Robin, then Priority, then SJF, then FCFS) that uses the same observation as the learner and has no fitted parameters.
- **Status:** implemented and covered by automated tests

M6 — Experiment orchestration, metrics and reporting

- **Files:** experiments/runtime_experiment.py, experiments/runtime_config.py, evaluation/metrics.py
- **Contract:** configuration and trained models → CSV and JSON artifacts, paired intervals, markdown report
- **Responsibility:** Trains five agents, validates, checks fingerprint disjointness, runs the held-out test, computes paired crossed-bootstrap intervals and writes every artifact.
- **Status:** implemented and covered by automated tests

4. Workflow diagram for each major module

Each module has its own Mermaid flowchart, followed by its numbered steps. Diamonds are decision points; the numbers in the diagram match the numbered steps.

M1. Workload generator and dataset builder

Source: workload/generator.py, workload/models.py

Diagram 3 — Workflow of M1, workload generator and dataset builder

```mermaid
flowchart TD
    N1["1. Select family configuration"]
    N2["2. Derive the workload seed"]
    N3["3. Sample 15 processes"]
    N4{"4. Valid burst, arrival and priority values?"}
    N5["5. Fingerprint and tag the workload"]
    N6["6. Assemble the split"]
    X1(["Reject workload: ValidationError"])
    N1 --> N2
    N2 --> N3
    N3 --> N4
    N4 -->|"yes"| N5
    N4 -->|"no"| X1
    N5 --> N6
```

1. **Select family configuration —** one of seven frozen parameter sets (burst, arrival, priority).
2. **Derive the workload seed —** SHA-256 of (master seed, split tag, family index, repetition).
3. **Sample 15 processes —** burst, arrival time and priority drawn by the family rules.
4. **Decision: valid values? —** burst > 0, arrival >= 0, priority >= 0, unique PIDs. If not, the workload is rejected with a ValidationError.
5. **Fingerprint and tag —** SHA-256 workload fingerprint plus split, family and repetition metadata.
6. **Assemble the split —** 6,000 training episodes, 140 validation and 210 held-out test workloads.

M2. Discrete-event runtime environment

Source: scheduler/runtime.py (one pass per decision epoch)

Diagram 4 — Workflow of M2, discrete-event runtime environment

```mermaid
flowchart TD
    N1["1. Advance to the next decision epoch"]
    N2["2. Admit arrivals up to the epoch"]
    N3["3. Freeze the causal observation"]
    N4["4. Obtain the policy from the controller"]
    N5{"5. Selected PID differs from the running PID?"}
    N6["6. Charge one switch-cost unit"]
    N7["7. Execute the segment"]
    N8["8. Settle the endpoint"]
    N9["9. Return interval reward r = -dW / q"]
    N10{"10. All processes completed?"}
    X1(["Emit schedule trace and six metrics"])
    N1 --> N2
    N2 --> N3
    N3 --> N4
    N4 --> N5
    N5 -->|"yes"| N6
    N5 -->|"no, or first dispatch"| N7
    N6 --> N7
    N7 --> N8
    N8 --> N9
    N9 --> N10
    N10 -->|"no"| N1
    N10 -->|"yes"| X1
```

1. **Advance to the next decision epoch —** a dispatch point, a completion or a Round-Robin quantum boundary.
2. **Admit arrivals —** every arrival at or before the epoch, in (arrival time, pid) order.

3. **Freeze the causal observation —** arrived and ready work only; future processes stay private.
4. **Obtain the policy —** FCFS, SJF, Round Robin or Priority from the active controller.
5. **Decision: running PID changes? —** if the selected PID differs from the running PID, go to step 6; otherwise, including the first dispatch, go to step 7.
6. **Charge the switch cost —** one unit of simulated time before the segment starts.
7. **Execute the segment —** min(quantum, remaining burst) for Round Robin; otherwise to completion.
8. **Settle the endpoint —** admit arrivals first; a completed process leaves; an unfinished Round-Robin process is appended to the tail.
9. **Return the interval reward —** r = -dW / max(1, q), where dW is the waiting-time area accumulated since the previous decision.
10. **Decision: all processes completed? —** if yes, emit the schedule trace and six metrics; if no, return to step 1.

M3. Causal observation and state encoder

Source: rl/runtime_state.py

Diagram 5 — Workflow of M3, causal observation and state encoder

```mermaid
flowchart TD
    N1["1. Receive the RuntimeObservation"]
    N2["2. Extract five causal features"]
    N3["3. Discretise each feature relative to the quantum"]
    N4["4. Flatten the bins in mixed-radix order"]
    N5["5. Return state index s in [0, 162)"]
    N1 --> N2
    N2 --> N3
    N3 --> N4
    N4 --> N5
```

1. **Receive the observation —** an immutable RuntimeObservation; any other object, and an empty ready queue, are rejected.
2. **Extract five causal features —** ready count, completed count, median ready remaining burst, mean ready arrival age and arrived-work priority spread.
3. **Discretise each feature —** bin edges are expressed relative to the quantum (q = 4).
4. **Flatten in mixed-radix order —** 3 x 3 x 3 x 3 x 2 collapses to one integer.
5. **Return the state index —** s in [0, 162), used by the controller only.

M4. Sequential Q-learning controller

Source: rl/runtime_controller.py, rl/q_learning.py

Diagram 6 — Workflow of M4, sequential Q-learning controller

```mermaid
flowchart TD
    N1["1. Encode the observation: s = encode(obs)"]
    N2{"2. Training run?"}
    N3{"3. First visit to s, or random draw below epsilon?"}
    N4["4. Choose a uniformly random action"]
    N5["5. Choose the greedy action over visited actions"]
    N6{"6. Any visited action in s?"}
    N7["7. Count an unseen-state fallback to the heuristic"]
    N8["8. Choose the greedy action over visited actions"]
    N9["9. Receive reward r = -dW / q"]
    N10["10. Update Q(s,a) from visited next-state actions"]
    N11["11. Persist Q values and visit counts"]
    X1(["Return action (evaluation, read-only)"])
    N1 --> N2
    N2 -->|"yes"| N3
    N2 -->|"no: evaluation"| N6
    N3 -->|"yes"| N4
    N3 -->|"no"| N5
    N6 -->|"yes"| N8
    N6 -->|"no"| N7
    N7 --> X1
    N8 --> X1
    N4 --> N9
    N5 --> N9
    N9 --> N10
    N10 --> N11
```

1. **Encode the observation —** s = encode(observation).
2. **Decision: training run? —** training uses the exploration rule below; evaluation is greedy and read-only.
3. **Decision: first visit or exploration? —** a first visit to s, or a draw below epsilon, gives a random action.
4. **Random action —** a uniformly random action out of four.
5. **Greedy action —** the highest-valued action among those already visited in s.
6. **Decision: any visited action in s? —** if none, the fallback in step 7 applies.
7. **Counted fallback —** use the predeclared heuristic and count the decision as an unseen-state fallback.
8. **Greedy action in evaluation —** the highest-valued visited action; no update is made.
9. **Receive the reward —** r = -dW / q from the environment.
10. **Update the table —** Q(s,a) += alpha * (r + gamma * max over visited a' of Q(s',a') - Q(s,a)); terminal transitions omit the bootstrap.
11. **Persist values and visits —** Q values and visit counts are written per training seed.

M5. Predeclared heuristic controller

Source: rl/runtime_heuristic.py (fixed rules, no learning)

Diagram 7 — Workflow of M5, predeclared heuristic controller

```mermaid
flowchart TD
    N1["1. Receive the same frozen observation"]
    N2{"2. At least 3 ready jobs and mean arrival age >= quantum?"}
    N3{"3. At least 2 ready jobs with distinct priorities?"}
    N4{"4. Largest remaining burst >= 2 x smallest?"}
    X1(["Return Round Robin"])
    X2(["Return Priority"])
    X3(["Return SJF"])
    X4(["Return FCFS (default)"])
    N1 --> N2
    N2 -->|"yes"| X1
    N2 -->|"no"| N3
    N3 -->|"yes"| X2
    N3 -->|"no"| N4
    N4 -->|"yes"| X3
    N4 -->|"no"| X4
```

1. **Receive the observation —** the same frozen RuntimeObservation that the learner receives.
2. **Round Robin rule —** at least three ready jobs and mean arrival age >= one quantum: return Round Robin.
3. **Priority rule —** otherwise, at least two ready jobs with distinct priorities: return Priority.
4. **SJF rule —** otherwise, largest remaining burst >= 2 x smallest: return SJF.
5. **Default —** otherwise return FCFS. The rule is deterministic, with no state and no Q update, and is identical in training and evaluation.

M6. Experiment orchestration, metrics and reporting

Source: experiments/runtime_experiment.py, evaluation/metrics.py

Diagram 8 — Workflow of M6, experiment orchestration, metrics and reporting

```mermaid
flowchart TD
    N1["1. Train five agents on six families"]
    N2["2. Validate on 140 workloads"]
    N3{"3. Training and validation fingerprints disjoint?"}
    N4["4. Select the illustrative trace by the validation-only rule"]
    N5["5. Generate the 210 held-out test workloads"]
    N6{"6. Validation and test fingerprints disjoint, and training disjoint from both?"}
    N7["7. Run the held-out test and derive six metrics per workload"]
    N8["8. Compare pairwise: paired differences and crossed bootstrap"]
    N9["9. Write CSV, JSON, Q-table, coverage and report artifacts"]
    X1(["Stop: ValidationError"])
    N1 --> N2
    N2 --> N3
    N3 -->|"yes"| N4
    N3 -->|"no"| X1
    N4 --> N5
    N5 --> N6
    N6 -->|"yes"| N7
    N6 -->|"no"| X1
    N7 --> N8
    N8 --> N9
```

1. **Train five agents —** seeds 7101–7105; 1,200 sequential episodes each over six families, with poisson_arrivals withheld.
2. **Validate —** 140 workloads (seed 8201), used for reporting and for the demo rule only, never for tuning.
3. **Decision: training and validation disjoint? —** zero fingerprint overlap is required; otherwise the run stops.

4. **Select the illustrative trace —** by the validation-only rule, before any test workload is generated.
5. **Generate the held-out test set —** 210 untouched workloads (seed 19301), generated only after steps 1 to 4.
6. **Decision: all splits disjoint? —** validation and test must not overlap, and training must not overlap either; otherwise the run stops.
7. **Run the held-out test —** greedy, read-only evaluation; six metrics per workload derived from the traces.
8. **Compare pairwise —** paired per-workload differences with a family-stratified crossed bootstrap of 1,000 replicates (seed 104729).
9. **Write the artifacts —** CSV tables, Q table, state-action coverage, manifest, demo trace, summary JSON and the generated report.

5. Dataset details and data collection procedure

5.1 Data collection procedure

No external or personal data is used. The dataset is synthetic and generated entirely by the project's own generator, so it can be reproduced from the declared integers on any machine.

- **D1:** Fix seven family configurations — burst distribution and range, arrival pattern and window or rate, and priority pattern, declared once in config.py and frozen in the runtime experiment configuration.
- **D2:** Derive one seed per workload — workload seed = SHA-256(master seed, split tag, family index, repetition or episode); no global random state is shared between workloads.
- **D3:** Generate 15 processes per workload — burst, arrival time and priority are drawn by the family rules, then validated (positive burst, non-negative arrival, priority within bounds).
- **D4:** Assign workloads to splits — training: 1,200 sequential episodes for each of five seeds, which is 6,000 episodes in total. Each episode generates one new workload, and the six training families cycle in a fixed order, so each family receives 200 episodes per seed. Validation: 140 workloads (7 families x 20). Held-out test: 210 workloads (7 families x 30).
- **D5:** Fingerprint and check disjointness — each workload gets a SHA-256 fingerprint; the pipeline asserts zero overlap between training, validation and test before the held-out test runs.
- **D6:** Store only what is needed — metrics, the manifest (split, family, seed, fingerprint), Q tables and summaries are committed; the large per-decision event logs are regenerated by the command and gitignored.
- **D7:** Regenerate on demand — python main.py runtime-experiment rebuilds every workload and every metric from the declared seeds.

5.2 Dataset inventory

- **Training (sequential episodes) —** six training families; 1,200 episodes per seed, five seeds; 6,000 workloads; 90,000 processes; master seed 7101–7105.
- **Validation —** all seven families; 20 repetitions per family; 140 workloads; 2,100 processes; master seed 8201.
- **Held-out test —** all seven families; 30 repetitions per family; 210 workloads; 3,150 processes; master seed 19301.

5.3 Family specification

Each family generates 15 processes. Parameters are taken from the frozen configuration (the same values are stored in results/runtime/runtime_summary.json).

- **short_jobs —** 80% of bursts uniform over 1–5 and the rest uniform over 6–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5. Role: training and evaluation.
- **long_jobs —** 20% of bursts uniform over 1–5 and the rest uniform over 6–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5. Role: training and evaluation.
- **mixed —** Bursts uniform over 1–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5. Role: training and evaluation.
- **cpu_bursty —** Bursts uniform over 20–50; arrival times uniform over the integers 0–15; priorities uniform over 1–5. Role: training and evaluation.
- **priority_skewed —** Bursts uniform over 1–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5, except that with probability 0.7 the priority is drawn from 1–2 instead. Role: training and evaluation.
- **staggered_interactive —** One long process with burst uniform over 25–50 at time 0, and 14 short processes with bursts uniform over 1–3; first arrival at time 0, then integer gaps uniform over 4–6; all priorities equal to 1. Role: training and evaluation.
- **poisson_arrivals —** Bursts uniform over 1–20; arrivals from a Poisson process with rate 0.5 (exponential inter-arrival times, floored to whole time units), first arrival at time 0; priorities uniform over 1–5. Role: evaluation only; never used for training.

5.4 Measured dataset statistics

The statistics below were recomputed with the project's generator at document-build time, so they describe the actual held-out dataset rather than the configuration alone. High priority means priority <= 2.

- **short jobs —** 30 workloads, 450 processes; burst 8.5 ± 12.1; mean arrival 9.6; mean total work 128; high-priority share 40%.
- **long jobs —** 30 workloads, 450 processes; burst 22.2 ± 15.4; mean arrival 9.6; mean total work 333; high-priority share 41%.
- **mixed —** 30 workloads, 450 processes; burst 24.7 ± 14.3; mean arrival 10.0; mean total work 370; high-priority share 41%.
- **cpu bursty —** 30 workloads, 450 processes; burst 35.0 ± 9.0; mean arrival 7.8; mean total work 525; high-priority share 38%.
- **priority skewed —** 30 workloads, 450 processes; burst 26.3 ± 14.6; mean arrival 10.0; mean total work 395; high-priority share 82%.
- **staggered interactive —** 30 workloads, 450 processes; burst 4.3 ± 9.1; mean arrival 35.4; mean total work 65; high-priority share 100%.
- **poisson arrivals —** 30 workloads, 450 processes; burst 10.7 ± 5.6; mean arrival 13.5; mean total work 160; high-priority share 44%.
- **All families —** 210 workloads, 3,150 processes; burst 18.8 ± 15.8; mean arrival 13.7; mean total work 282; high-priority share 55%.

5.5 Data integrity and assumptions

- Reproducibility — seeds are derived with SHA-256, so a workload is a pure function of (master seed, split tag, family index, repetition).

- Disjointness — the pipeline stops if a fingerprint appears in two splits; the committed run reports overlap = false over 6,000 distinct training fingerprints.
- Burst knowledge — exact burst lengths are known once a process arrives. This is the assumption already required by standalone SJF; no burst predictor is included.
- Storage — per-decision event logs are large and are regenerated by the command and gitignored; metrics, Q tables and manifests are committed.
- Licensing and privacy — the dataset is entirely synthetic, so no personal or licensed data is involved.

6. Tools, technologies, algorithms and frameworks used

6.1 Tools and technologies

- **Language:** Python 3.11.2 — whole implementation, command-line interface and report builders.
- **Numerics:** NumPy 2.4.6 — Q table and visit counts stored as arrays; bootstrap resampling.
- **Data handling:** pandas 3.0.6 — metric aggregation and CSV artifact export.
- **Testing:** pytest 9.1.1 — automated checks of every module (section 8.3).
- **Diagrams:** Mermaid (text source) — all diagrams in this document; renders in GitHub, VS Code, Obsidian and mermaid.live.
- **Document generation:** python-docx 1.2.0 — renders this text-only DOCX.
- **PDF generation:** ReportLab 5.0.1 — renders the text-only PDF from the DOCX.
- **Hashing:** hashlib (SHA-256, standard library) — seed derivation and workload fingerprints.
- **Standard library:** argparse, dataclasses, json, pathlib — configuration objects, command-line interface and artifact serialisation.
- **Version control:** Git and GitHub — branch-per-change workflow with pull requests.
- **Environment:** Linux 6.1 (x86-64) with venv and pip — development, experiment runs and report builds.

No deep-learning framework is used. The controller is a tabular Q-learning agent, so every value, visit count and fallback is inspectable in the committed artifacts.

6.2 Algorithms

- **FCFS (action 0) —** dispatch the head of the ready queue, non-preemptively. Used for: baseline and runtime action; after a Round-Robin requeue it follows live queue order.
- **SJF (action 1) —** smallest remaining burst first, non-preemptive, ties in ready-queue order. Used for: baseline and runtime action; minimises mean waiting time when all jobs are present together, and is a strong but not guaranteed-optimal reference under staggered arrivals.
- **Round Robin (action 2) —** head of the queue runs for min(quantum, remaining burst); unfinished jobs go to the tail after endpoint arrivals. Used for: baseline and runtime action; bounds the time to first service.
- **Priority (action 3) —** highest static priority first, non-preemptive, ties in ready-queue order. Used for: baseline and runtime action; lower numeric value means higher priority.
- **Tabular Q-learning —** Q(s,a) += alpha * (r + gamma * max over visited actions of Q(s',.) - Q(s,a)) with epsilon-greedy exploration and unvisited-action masking. Used for: the learned runtime controller over 162 states x 4 actions.

- **Predeclared heuristic —** Round Robin, then Priority, then SJF, then FCFS, applied to the same observation. Used for: non-learning adaptive reference.
- **Crossed bootstrap —** resample workloads within each family and resample model seeds independently; 1,000 replicates. Used for: paired 95% intervals for every comparison.
- **Discrete-event simulation —** advance the event list with a persistent FIFO ready queue and switch-cost charging. Used for: the machine being controlled.

6.3 Controller pseudocode

```
SELECT_ACTION(obs):                 # training: epsilon-greedy; evaluation: greedy
    s <- encode(obs)                # 5 causal features -> state in [0, 162)
    if evaluating:
        if no visited action in s:  # wholly unseen state
            fallback_count += 1
            return HEURISTIC(obs)   # predeclared rule, reported rather than hidden
        return argmax over visited actions of Q[s, a]
    if first_visit(s) or rand() < epsilon(episode):
        return uniform_random_action()
    return argmax over visited actions of Q[s, a]

UPDATE(s, a, r, s', terminal):
    target <- r if terminal else r + gamma * max over visited actions of Q[s', .]
    Q[s, a] <- Q[s, a] + alpha * (target - Q[s, a])
    visits[s, a] <- visits[s, a] + 1

HEURISTIC(obs):                     # predeclared, no learning
    if ready_count >= 3 and mean_arrival_age >= quantum:  return Round Robin
    if at least 2 ready jobs have distinct priorities:    return Priority
    if max(remaining_burst) >= 2 * min(remaining_burst):  return SJF
    return FCFS
```

7. Experimental plan and evaluation metrics

7.1 Experimental plan

- **P1:** Fix the design before looking at results — families, splits, seeds, quantum (4), switch cost (1), learning rate and the heuristic rule are declared in configuration and committed.
- **P2:** Train five independent agents — seeds 7101–7105; 1,200 sequential episodes each over six families, with poisson_arrivals withheld.
- **P3:** Validate — 140 workloads (seed 8201), used only for reporting and for the illustrative-trace rule.
- **P4:** Test once, held out — 210 untouched workloads (seed 19301). The fixed policies and the heuristic are deterministic and run once each; the five Q-learning models are evaluated greedily and read-only.
- **P5:** Compare every method against the others — paired per-workload differences, plus independent resampling of the five model seeds.
- **P6:** Quantify uncertainty — family-stratified crossed bootstrap, 1,000 replicates, seed 104729; 95% intervals.
- **P7:** Publish artifacts — metric tables, Q values, visit counts, coverage, manifest, demo trace and summary JSON, written to results/runtime/ by one command.

7.2 Evaluation metrics

- **Mean waiting time —** mean over processes of (turnaround time minus burst time), that is, time spent waiting in the ready queue. Why: primary metric; it is the quantity the reward accumulates.

- **Mean turnaround time —** mean over processes of (completion time minus arrival time). Why: end-to-end latency experienced by a job.
- **Mean response time —** mean over processes of (first dispatch time minus arrival time). Why: interactivity; this is what Round Robin improves.
- **CPU utilisation (%) —** busy time divided by total elapsed time, times 100. Why: the cost of switching and idling.
- **Throughput —** processes completed per unit of simulated time. Why: work completed per unit time.
- **Context switches —** number of changes of the running PID. Why: overhead proxy; the switch cost is charged for every method.
- **Policy switches per trace —** decisions where the chosen policy differs from the previous one. Why: how much the controller actually adapts.
- **State-action coverage —** visited (state, action) pairs out of 648, and unseen-state fallbacks. Why: whether the learned table is populated where it is used.

7.3 Statistical procedure

Comparisons are paired: for each workload, the difference (target minus reference) is computed on the same workload. For Q-learning, the model seed is also resampled independently of the workloads (a crossed bootstrap). Resampling is stratified by workload family, with 1,000 replicates drawn using seed 104729. An interval that excludes zero on the favourable side is treated as a real difference on this benchmark; an interval containing zero is reported as inconclusive.

7.4 Results obtained with this plan

- **FCFS —** waiting 127.797 ± 79.81; turnaround 146.612 ± 90.51; response 127.797 ± 79.81; CPU utilisation 92.24 ± 6.15 %; throughput 0.077 ± 0.056; context switches 14.00 ± 0.00; policy switches per trace 0.00.
- **SJF —** waiting 93.345 ± 68.78; turnaround 112.161 ± 79.39; response 93.345 ± 68.78; CPU utilisation 92.24 ± 6.15 %; throughput 0.077 ± 0.056; context switches 14.00 ± 0.00; policy switches per trace 0.00.
- **Round Robin —** waiting 215.729 ± 172.82; turnaround 234.544 ± 183.45; response 26.220 ± 9.15; CPU utilisation 78.22 ± 1.90 %; throughput 0.068 ± 0.054; context switches 74.77 ± 40.70; policy switches per trace 0.00.
- **Priority —** waiting 129.207 ± 80.81; turnaround 148.022 ± 91.53; response 129.207 ± 80.81; CPU utilisation 92.24 ± 6.15 %; throughput 0.077 ± 0.056; context switches 14.00 ± 0.00; policy switches per trace 0.00.
- **Causal heuristic —** waiting 203.404 ± 159.23; turnaround 222.220 ± 169.85; response 40.070 ± 17.05; CPU utilisation 80.48 ± 2.22 %; throughput 0.071 ± 0.058; context switches 67.37 ± 39.49; policy switches per trace 3.21.
- **Runtime Q-learning —** waiting 104.789 ± 71.43; turnaround 123.604 ± 81.86; response 86.341 ± 59.82; CPU utilisation 90.84 ± 7.04 %; throughput 0.076 ± 0.054; context switches 17.32 ± 4.66; policy switches per trace 4.73.

All values are means ± standard deviation across the 210 held-out workloads (master seed 19301), with quantum 4 and switch cost 1. Q-learning also averages the five model seeds.

Diagram 9 — Mean waiting time by method, held-out test (section 7.4)

```mermaid
xychart-beta
    title "Mean waiting time by method - 210 held-out workloads"
    x-axis ["FCFS", "SJF", "RR", "Priority", "Heuristic", "Runtime Q"]
    y-axis "time units" 0 --> 237
    bar [127.797, 93.345, 215.729, 129.207, 203.404, 104.789]
```

Diagram 10 — Mean response time by method, held-out test (section 7.4)

```mermaid
xychart-beta
    title "Mean response time by method - 210 held-out workloads"
    x-axis ["FCFS", "SJF", "RR", "Priority", "Heuristic", "Runtime Q"]
    y-axis "time units" 0 --> 142
    bar [127.797, 93.345, 26.22, 129.207, 40.07, 86.341]
```

7.5 Paired comparisons

Each line gives target minus reference, with its 95% crossed-bootstrap interval, as committed in results/runtime/runtime_paired_comparisons.csv. Turnaround differences equal waiting differences exactly, because both methods serve the same bursts on each workload, and throughput is in the CSV.

- **Runtime Q minus SJF —** waiting time: difference +11.443, 95% interval [+5.022, +19.297], favours the reference.
- **Runtime Q minus SJF —** response time: difference -7.004, 95% interval [-9.541, -4.305], favours the target.
- **Runtime Q minus SJF —** context switches: difference +3.322, 95% interval [+2.209, +4.555], favours the reference.
- **Runtime Q minus SJF —** CPU utilisation (percentage points): difference -1.402, 95% interval [-1.843, -1.005], favours the reference.
- **Runtime Q minus Heuristic —** waiting time: difference -98.616, 95% interval [-106.097, -90.192], favours the target.
- **Runtime Q minus Heuristic —** response time: difference +46.272, 95% interval [+43.472, +49.100], favours the reference.
- **Runtime Q minus Heuristic —** context switches: difference -50.050, 95% interval [-51.929, -48.066], favours the target.
- **Runtime Q minus Heuristic —** CPU utilisation (percentage points): difference +10.354, 95% interval [+9.890, +10.834], favours the target.
- **RR minus SJF —** waiting time: difference +122.383, 95% interval [+117.619, +127.315], favours the reference.
- **RR minus SJF —** response time: difference -67.126, 95% interval [-70.219, -64.285], favours the target.
- **RR minus SJF —** context switches: difference +60.767, 95% interval [+58.995, +62.491], favours the reference.
- **RR minus SJF —** CPU utilisation (percentage points): difference -14.021, 95% interval [-14.241, -13.760], favours the reference.
- **Heuristic minus SJF —** waiting time: difference +110.059, 95% interval [+106.219, +114.032], favours the reference.
- **Heuristic minus SJF —** response time: difference -53.276, 95% interval [-56.026, -50.873], favours the target.

- **Heuristic minus SJF —** context switches: difference +53.371, 95% interval [+51.666, +55.191], favours the reference.
- **Heuristic minus SJF —** CPU utilisation (percentage points): difference -11.756, 95% interval [-11.992, -11.518], favours the reference.
- **FCFS minus SJF —** waiting time: difference +34.451, 95% interval [+32.640, +36.540], favours the reference.
- **FCFS minus SJF —** response time: difference +34.451, 95% interval [+32.605, +36.440], favours the reference.
- **FCFS minus SJF —** context switches: difference +0.000, 95% interval [+0.000, +0.000], no difference.
- **FCFS minus SJF —** CPU utilisation (percentage points): difference +0.000, 95% interval [+0.000, +0.000], no difference.
- **Priority minus SJF —** waiting time: difference +35.862, 95% interval [+33.987, +37.684], favours the reference.
- **Priority minus SJF —** response time: difference +35.862, 95% interval [+33.956, +37.823], favours the reference.
- **Priority minus SJF —** context switches: difference +0.000, 95% interval [+0.000, +0.000], no difference.
- **Priority minus SJF —** CPU utilisation (percentage points): difference +0.000, 95% interval [+0.000, +0.000], no difference.

7.6 What the plan has established so far

- Against Round Robin, mean waiting time is lower by 110.940 time units (104.789 against 215.729). This is the difference of the two committed comparisons against SJF (Q minus SJF +11.443; RR minus SJF +122.383). No paired interval was computed for this pair. FCFS and Priority lie between the two, at +34.451 and +35.862 relative to SJF.
- Against the predeclared adaptive rule, mean waiting time is lower by 98.616 (paired difference Q minus rule -98.616; 95% interval [-106.097, -90.192]). Learning therefore improves on this particular adaptive rule on this benchmark. The rule has the lower response time: Q-learning's response time is higher by 46.272, because the rule selects Round Robin whenever three or more aged jobs are ready.
- Against SJF the result is mixed. Waiting time is higher for Q-learning by 11.443 (95% interval 5.022 to 19.297), while response time is lower by 7.004 (95% interval 4.305 to 9.541). Non-preemptive SJF with exact bursts and no mid-job switch cost is a strong reference on this benchmark, and the learned controller does not beat it on waiting time.
- Q-learning's mean waiting time lies between the SJF and Round Robin values on all 7 held-out families, including the unseen poisson_arrivals family (68.91, against SJF 49.76 and Round Robin 113.58).
- Controller overhead is small relative to simulated time: about 13.69 microseconds to build an observation and 21.24 microseconds to select an action per decision on the build host. These timings are measured separately and are never charged to the schedule.

8. Implementation progress

Review 2 requires at least 50% of the proposed work to be complete, with the completed modules demonstrated by valid outputs. All six proposed modules are implemented, tested and demonstrated below, which is 100% of the proposed Review 2 scope. Section 9.2 lists work planned beyond that scope.

8.1 Completion by module

- **M1 — Workload generator and dataset builder.** completion 100%; status: implemented and tested; evidence: section 8.2.
- **M2 — Discrete-event runtime environment.** completion 100%; status: implemented and tested; evidence: section 8.2.
- **M3 — Causal observation and state encoder.** completion 100%; status: implemented and tested; evidence: section 8.2.
- **M4 — Sequential Q-learning controller.** completion 100%; status: implemented and tested; evidence: section 8.2.
- **M5 — Predeclared heuristic controller.** completion 100%; status: implemented and tested; evidence: section 8.2.
- **M6 — Experiment orchestration, metrics and reporting.** completion 100%; status: implemented and tested; evidence: section 8.2.

Overall completion of the proposed Review 2 modules: 6 of 6 (100%).

8.2 Outputs demonstrated by each module

Each module is demonstrated by the output it produced in the committed run:

- **M1 —** 6,350 manifest rows; 210 held-out workloads (3,150 processes) regenerated from the declared seeds, with the measured burst, arrival and priority statistics of section 5
- **M2 —** 2,100 final-test metric rows produced from simulated traces; fixed-action runs reproduce the standalone FCFS, SJF, Round Robin and Priority schedules exactly in the test suite
- **M3 —** 162-state encoder; 81-89 distinct states visited per agent during training
- **M4 —** 3,240 Q values stored; 267-276 of 648 state-action pairs visited; 1-3 counted unseen-state fallbacks per model on the held-out test
- **M5 —** heuristic rows present for all 210 held-out workloads; mean waiting 203.404 with 3.21 policy switches per trace
- **M6 —** 36 paired comparisons with 95% bootstrap intervals, the coverage table, the learned-switch demo trace and the generated runtime_report.md

8.3 Demonstration 1 — automated verification

The test suite collects 273 checks. Counts per test file, recorded during this build:

- **test_adaptive.py:** 18 checks — legacy offline selector: state encoding, decisions, read-only evaluation.
- **test_experiment_design.py:** 4 checks — frozen seeds, split sizes and configuration invariants.
- **test_experiments.py:** 30 checks — legacy experiment pipeline, metrics and report generation.
- **test_fcfs.py:** 11 checks — FCFS ordering, idle handling and metric computation.
- **test_generator.py:** 7 checks — workload family generation and reproducibility from seeds.
- **test_metrics.py:** 14 checks — waiting, turnaround, response, utilisation and throughput definitions.
- **test_priority.py:** 14 checks — priority ordering, ties and non-preemption.
- **test_q_learning.py:** 29 checks — Q updates, epsilon schedule, masking, visit counts, evaluation.

- **test_reward.py:** 16 checks — reward composition and clipping against the reference policies.
- **test_round_robin.py:** 16 checks — quantum boundaries, re-enqueue order and completion behaviour.
- **test_runtime_experiment.py:** 4 checks — split disjointness, determinism, artifact regeneration.
- **test_runtime_scheduler.py:** 23 checks — runtime environment: fixed-action equivalence, causality, reward identity.
- **test_schedule_result.py:** 14 checks — schedule trace validation and aggregate result objects.
- **test_sjf.py:** 12 checks — SJF selection, tie-breaking and non-preemption.
- **test_state.py:** 24 checks — offline state discretisation and mixed-radix indexing.
- **test_workload_models.py:** 37 checks — process and workload models, validation and trace invariants.

Output of python -m pytest -q:

```
$ python -m pytest -q
273 passed in 9.63s
```

8.4 Demonstration 2 — end-to-end run output

The full pipeline (training, validation, held-out testing and artifact export) runs with one command. The output below reproduces the lines that command prints, with file paths shortened to file names, taken from the committed artifacts. A fresh run of the command on the build host printed the same metric values; its schedule metrics and policy decisions match the committed artifacts exactly, and only the per-decision timing columns depend on the host.

Output of python main.py runtime-experiment:

```
$ python main.py runtime-experiment
Causal event-driven runtime experiment
--------------------------------------
  training seeds                 : 7101, 7102, 7103, 7104, 7105
  training episodes/model        : 1200
  validation workloads           : 140
  final held-out test workloads  : 210
  final-test mean waiting time:
    Causal heuristic      : 203.404
    FCFS                  : 127.797
    Priority              : 129.207
    Round Robin           : 215.729
    Runtime Q-learning    : 104.789
    SJF                   : 93.345
  learned within-trace switches : 15 (23 decisions; runtime_learned_switch_demo.json)
  artifacts:
    training_metrics        : runtime_training_metrics.csv
    validation_metrics      : runtime_validation_metrics.csv
    final_test_metrics      : runtime_final_test_metrics.csv
    validation_decisions    : runtime_validation_decisions.csv
    final_test_decisions    : runtime_final_test_decisions.csv
    workload_manifest       : runtime_workload_manifest.csv
    state_action_coverage   : runtime_state_action_coverage.csv
    q_table                 : runtime_q_table.csv
    paired_comparisons      : runtime_paired_comparisons.csv
    final_test_summary      : runtime_final_test_summary.csv
    validation_summary      : runtime_validation_summary.csv
    summary                 : runtime_summary.json
    learned_switch_demo     : runtime_learned_switch_demo.json
    report                  : runtime_report.md
```

8.5 Demonstration 3 — learned behaviour within one trace

The controller switches policies inside a single trace. The example comes from the validation split, chosen by the predeclared validation-only rule (maximise policy switches among traces with zero unseen-state fallbacks). It is training seed 7103, family staggered_interactive, repetition 0, fingerprint 275409d7458331f6. It contains 15 policy switches over 23 decisions, with mean waiting time 5.933 and mean response time 2.933. It illustrates the mechanism and is not a performance claim.

Diagram 11 — Illustrative validation trace (section 8.5). Each bar is one execution segment, labelled with the policy that ran it. Gaps between bars are the one-unit switch costs.

```mermaid
gantt
    title Illustrative validation trace - seed 7103, staggered_interactive repetition 0
    dateFormat X
    axisFormat %s
    section P1
    Round Robin :t1, 0, 8
    Round Robin :t2, 15, 19
    Round Robin :t3, 22, 26
    Round Robin :t4, 33, 37
    Round Robin :t5, 47, 51
    Round Robin :t6, 60, 64
    Round Robin :t7, 67, 71
    Round Robin :t8, 77, 92
    section P2
    FCFS :t9, 9, 11
    section P3
    SJF :t10, 12, 14
    section P4
    SJF :t11, 20, 21
    section P5
    SJF :t12, 27, 30
    section P6
    SJF :t13, 31, 32
    section P7
    SJF :t14, 38, 39
    section P8
    SJF :t15, 40, 43
    section P9
    SJF :t16, 44, 46
    section P10
    SJF :t17, 52, 53
    section P11
    SJF :t18, 54, 57
    section P12
    SJF :t19, 58, 59
    section P13
    SJF :t20, 65, 66
    section P14
    SJF :t21, 72, 73
    section P15
    SJF :t22, 74, 76
```

8.6 Artifacts produced

- **results/runtime/runtime_final_test_metrics.csv:** 2,100 rows — six metrics, switch and decision counts and timings on the held-out test.
- **results/runtime/runtime_validation_metrics.csv:** 1,400 rows — the same columns for the validation split.
- **results/runtime/runtime_training_metrics.csv:** 6,000 rows — per-episode sequential training outcomes.
- **results/runtime/runtime_paired_comparisons.csv:** 36 rows — paired differences with 95% crossed-bootstrap intervals.

- **results/runtime/runtime_state_action_coverage.csv:** 15 rows — visited states and pairs, fallback counts and timings per seed.
- **results/runtime/runtime_q_table.csv:** 3,240 rows — Q values and visit counts per training seed.
- **results/runtime/runtime_workload_manifest.csv:** 6,350 rows — split, family, seed and workload fingerprint for every workload.
- **results/runtime/runtime_learned_switch_demo.json:** 1 trace — observations, actions, switch times and execution trace of the illustrative run.
- **results/runtime/runtime_summary.json:** summary — configuration, split fingerprints, all means and software versions.
- **results/runtime/runtime_report.md:** report — generated markdown report of the run.

9. Challenges encountered and plan for completing the remaining work

9.1 Challenges encountered

- **Keeping the controller causal —** Why it mattered: an earlier whole-workload selector saw future arrivals and counterfactual schedules, so its results could not be read as a runtime controller. Resolution: introduced an immutable RuntimeObservation, made the encoder accept only that object, and added a paired hidden-future test that fails if unarrived processes change the observation or action prefix.
- **Round Robin and FCFS after a requeue —** Why it mattered: once Round Robin appends a preempted job to the tail, FCFS is ambiguous: original arrival order or live queue order. Resolution: defined one persistent FIFO ready queue with canonical (arrival time, pid) insertion; FCFS follows live queue order, and SJF and Priority ties preserve it; fixed-action runs are tested against the standalone schedulers.
- **Charging switch cost fairly —** Why it mattered: if the learner pays for switching but the baselines do not, the comparison is invalid; if nothing is charged, switching is unrealistic. Resolution: one unit is charged whenever the running PID changes, for every method including the fixed policies; arrivals during the switch are queued, but the chosen dispatch is not reconsidered.
- **A reward that matches the objective —** Why it mattered: a shaped reward can drift away from the metric being reported. Resolution: used r = -dW / q with gamma = 1, so the episodic return is exactly minus total waiting time divided by the quantum; a test asserts this identity.
- **Sparse coverage of the tabular space —** Why it mattered: only about 42% of the 648 state-action pairs are visited during training, and unseen states appear during evaluation. Resolution: kept the abstraction small, forced exploration on first visit, masked unvisited actions in greedy steps and bootstraps, and made unseen states fall back to the predeclared heuristic with the fallback count reported.
- **Reporting a negative result —** Why it mattered: the learner does not beat non-preemptive SJF on mean waiting time, so the result could be omitted or overstated. Resolution: reported as a headline finding with its bootstrap interval, and the structural reason stated: SJF receives exact bursts at arrival and pays no mid-job switch cost.
- **Host-dependent timing —** Why it mattered: wall-clock columns differ between machines, which can look like non-determinism. Resolution: observation, selection, Q-update and simulator times are stored in separate columns, never charged to simulated time, and excluded from every schedule comparison.

- **Large per-decision event logs —** Why it mattered: full causal decision logs are large and would bloat the repository. Resolution: the command writes them, they are gitignored, and they regenerate from the recorded configuration and seeds; compact metrics, Q tables and manifests are committed.

9.2 Plan for completing the remaining work

- **W1 — Sensitivity study.** gap: all results use quantum 4 and switch cost 1. Approach: re-run the plan over a grid of quanta and switch costs and report where the ordering changes. Deliverable: metric tables for each (quantum, cost) cell.
- **W2 — Burst prediction.** gap: exact bursts are assumed known at arrival. Approach: add an explicit predictor with measured error and re-run the controller under that noise. Deliverable: results with prediction-error bands.
- **W3 — Richer state abstraction.** gap: only about 42% of pairs are visited, and 162 states is coarse. Approach: tile coding or a small function approximator, keeping the same causal observation contract and fallback reporting. Deliverable: coverage and metric comparison against the tabular agent.
- **W4 — Multi-objective reward.** gap: waiting time is optimised, response time is not. Approach: add a constrained or weighted objective and measure the waiting-response trade-off. Deliverable: Pareto front over the two metrics.
- **W5 — I/O and blocking.** gap: processes never block, so the queue is CPU-bound only. Approach: model I/O bursts and blocked and wake-up transitions in the environment. Deliverable: extended environment with re-baselined comparisons.
- **W6 — Multicore extension.** gap: single CPU only. Approach: extend to N cores with per-core queues and a migration cost, choosing a policy per core. Deliverable: multicore metrics and overhead analysis.
- **W7 — Trace-driven validation.** gap: all workloads are synthetic. Approach: replay a public scheduling trace through the same environment and report the gap to the synthetic results. Deliverable: trace-driven result table.

9.3 Sequence for the remaining work

1. First, W1 (sensitivity) and W2 (burst prediction): they test whether the current conclusions depend on fixed parameters or on the idealised burst assumption.
2. Second, W3 and W4: they address the two measured weaknesses, coarse coverage and a single objective.
3. Third, W5 and W6: environment extensions that make the simulator more realistic, each with re-baselined comparisons.
4. Finally, W7: external validation on a real trace is the strongest available check that the synthetic ordering carries over.